

§ 4.3 栈的典型应用

4.3.1 逆序输出

在栈所擅长解决的典型问题中，有一类具有以下共同特征：首先，虽有明确的算法，但其解答却以线性序列的形式给出；其次，无论是递归还是迭代实现，该序列都是依逆序计算输出的；最后，输入和输出规模不确定，难以事先确定盛放输出数据的容器大小。因其特有的“后进先出”特性及其在容量方面的自适应性，使用栈来解决此类问题可谓恰到好处。

■ 进制转换

考查如下问题：任给十进制整数 n ，将其转换为 λ 进制的表示形式。比如 $\lambda = 8$ 时有

$$12345_{(10)} = 30071_{(8)}$$

一般地，设 $n = (d_m \dots d_2 d_1 d_0)_{(\lambda)} = d_m \times \lambda^m + \dots + d_2 \times \lambda^2 + d_1 \times \lambda^1 + d_0 \times \lambda^0$

若记 $n_i = (d_m \dots d_{i+1} d_i)_{(\lambda)}$

则有 $d_i = n_i \% \lambda$ 和 $n_{i+1} = n_i / \lambda$

这一递推关系对应的计算流程如下。可见，其输出的确为长度不定的逆序线性序列。

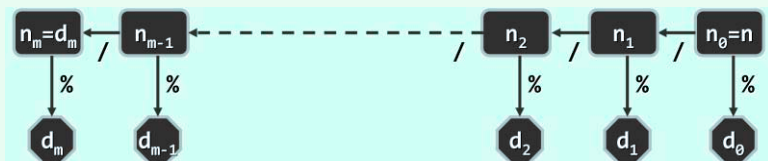


图4.4 进制转换算法流程

■ 递归实现

根据如图4.4所示的计算流程，可得到如代码4.2所示递归式算法。

```
1 void convert(Stack<char>& S, __int64 n, int base) { //十进制数n到base进制的转换（递归版）
2     static char digit[] //0 < n, 1 < base <= 16, 新进制下的数位符号，可视base取值范围适当扩充
3     = { '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F' };
4     if (0 < n) { //在尚有余数之前，不断
5         convert(S, n / base, base); //通过递归得到所有更高位
6         S.push(digit[n % base]); //输出低位
7     }
8 } //新进制下由高到低的各数位，自顶而下保存于栈S中
```

代码4.2 进制转换算法（递归版）

尽管新进制下的各数位须按由低到高次序逐位算出，但只要引入一个栈并将算得的数位依次入栈，则在计算结束后只需通过反复的出栈操作即可由高到低地将其顺序输出。

■ 迭代实现

这里的静态数位符号表在全局只需保留一份，但与一般的递归函数一样，该函数在递归调用栈中的每一帧都仍需记录参数 S 、 n 和 $base$ 。将它们改为全局变量固然可以节省这部分空间，但依然不能彻底地避免因调用栈操作而导致的空间和时间消耗。为此，不妨考虑改写为如代码4.3所示的迭代版本，既能充分发挥栈处理此类问题的特长，又可将空间消耗降至 $O(1)$ 。

```

1 void convert(Stack<char>& S, __int64 n, int base) { //十进制数n到base进制的转换（迭代版）
2     static char digit[] //0 < n, 1 < base <= 16, 新进制下的数位符号, 可视base取值范围适当扩充
3     = { '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', 'A', 'B', 'C', 'D', 'E', 'F' };
4     while (n > 0) { //由低到高, 逐一计算出新进制下的各数位
5         int remainder = (int) (n % base); S.push(digit[remainder]); //余数（当前位）入栈
6         n /= base; //n更新为其对base的除商
7     }
8 } //新进制下由高到低的各数位, 自顶而下保存于栈S中

```

代码4.3 进制转换算法（迭代版）

4.3.2 递归嵌套

具有自相似性的问题多可嵌套地递归描述, 但因分支位置和嵌套深度并不固定, 其递归算法的复杂度不易控制。栈结构及其操作天然地具有递归嵌套性, 故可用以高效地解决这类问题。以下先从混洗的角度介绍栈的递归嵌套性, 然后再讲解其在具体问题中的应用。

■ 栈混洗

考查三个栈A、B和S, 其中A含有n个元素, 自顶而下构成输入序列:

$$\{ a_1, a_2, \dots, a_n \}$$

B和S初始为空。若只允许通过S.push(A.pop())弹出栈A的顶元素并压入栈S中, 或通过B.push(S.pop())弹出S的顶元素并压入栈B中, 则在经过一系列这样的操作后, 当栈A和S均为空时, 原A中的元素应均已转入栈B。此时, 若将B中元素自底而上构成的序列记作:

$$\{ a_{k1}, a_{k2}, \dots, a_{kn} \}$$

则该序列称作原输入序列的一个栈混洗 (stack permutation)。

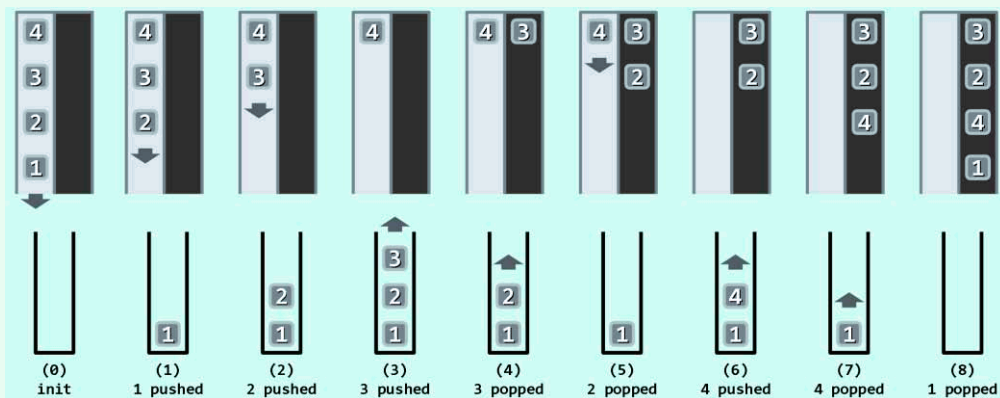


图4.5 栈混洗实例：从{ 1, 2, 3, 4 }到{ 3, 2, 4, 1 } (上方左侧为栈A, 右侧为栈B; 下方为栈S)

如图4.5所示, 设初始状态下栈A = { 1, 2, 3, 4 }, 栈S和B均为空; 经过“随机的”8次操作, A中的元素全部转入栈B中。此时, 栈B中元素自底而上对应的序列{ 3, 2, 4, 1 }, 即是原序列{ 1, 2, 3, 4 }的一个栈混洗。除了“实施出栈操作的栈不得为空”, 以上过程并无更多限制, 操作序列具有很大的随机性, 故同一输入序列的栈混洗并不唯一。比如就上例而言, { 1, 2, 3, 4 }、{ 4, 3, 2, 1 }以及{ 3, 2, 1, 4 }等也是栈混洗。

从图4.5也可看出,一般地对于长度为 n 的输入序列,每一栈混洗都对应于由栈 S 的 n 次push和 n 次pop构成的某一合法操作序列,比如{ 3, 2, 4, 1 }即对应于操作序列:

{ push, push, push, pop, pop, push, pop, pop }

反之,由 n 次push和 n 次pop构成的任何操作序列,只要满足“任一前缀中的push不少于pop”这一限制,则该序列也必然对应于某个栈混洗(习题[4-4])。

■ 括号匹配

对源程序的语法检查是代码编译过程中重要而基本的一个步骤,而对表达式括号匹配的检查则又是语法检查中必需的一个环节。其任务是,对任一程序块,判断其中的括号是否在嵌套的意义上完全匹配(简称匹配)。比如在以下两个表达式中,前者匹配,而后者不匹配。

$a / (b[i-1][j+1] + c[i+1][j-1]) * 2$

$a / (b[i-1][j+1]) + c[i+1][j-1] * 2$

■ 递归实现

不妨先只考虑圆括号。用 '+' 表示表达式的串接。

不难理解,一般地,若表达式 S 可分解为如下形式:

$S = S_0 + "(" + S_1 + ")" + S_2 + S_3$

其中 S_0 和 S_3 不含括号,且 S_1 中左、右括号数目相等,则 S 匹配当且仅当 S_1 和 S_2 均匹配。

按照这一理解,可采用分治策略设计算法如下:将表达式划分为子表达式 S_0 、 S_1 和 S_2 ,分别递归地判断 S_1 和 S_2 是否匹配。这一构思可具体实现如代码4.4所示。

```
1 void trim(const char exp[], int& lo, int& hi) { //删除表达式exp[lo, hi]不含括号的最长前缀、后缀
2     while ((lo <= hi) && (exp[lo] != '(') && (exp[lo] != '))) lo++; //查找第一个和
3     while ((lo <= hi) && (exp[hi] != '(') && (exp[hi] != '))) hi--; //最后一个括号
4 }
5
6 int divide(const char exp[], int lo, int hi) { //切分表达式exp[lo, hi],使exp匹配仅当它们匹配
7     int mi = lo; int crc = 1; //crc为[lo, mi]范围内左、右括号数目之差
8     while ((0 < crc) && (++mi < hi)) //逐个检查各字符,直到左、右括号数目相等,或者越界
9         { if (exp[mi] == ')') crc--; if (exp[mi] == '(') crc++; } //左、右括号分别计数
10    return mi; //若mi <= hi,则为合法切分点;否则,意味着局部不可能匹配
11 }
12
13 bool paren(const char exp[], int lo, int hi) { //检查表达式exp[lo, hi]是否括号匹配(递归版)
14    trim(exp, lo, hi); if (lo > hi) return true; //清除不含括号的前缀、后缀
15    if (exp[lo] != '(') return false; //首字符非左括号,则必不匹配
16    if (exp[hi] != ')') return false; //末字符非右括号,则必不匹配
17    int mi = divide(exp, lo, hi); //确定适当的切分点
18    if (mi > hi) return false; //切分点不合法,意味着局部以至整体不匹配
19    return paren(exp, lo + 1, mi - 1) && paren(exp, mi + 1, hi); //分别检查左、右子表达式
20 }
```

代码4.4 括号匹配算法(递归版)

其中, `trim()` 函数用于截除表达式中不含括号的头部和尾部, 即前缀 S_0 和后缀 S_3 。 `divide()` 函数对表达式做线性扫描, 并动态地记录已经扫描的左、右括号数目之差。如此, 当已扫过同样多的左、右括号时, 即确定了一个合适的切分点 mi , 并得到子表达式 $S_1 = \text{exp}(lo, mi)$ 和 $S_2 = \text{exp}(mi, hi]$ 。以下, 经递归地检查 S_1 和 S_2 , 即可判断原表达式是否匹配。

在最坏情况下 `divide()` 需要线性时间, 且递归深度为 $O(n)$, 故以上算法共需 $O(n^2)$ 时间。此外, 该方法也难以处理含有多种括号的表达式 (习题 [4-5] 和 [4-15]), 故有必要进一步优化。

■ 迭代实现

实际上, 只要将 `push`、`pop` 操作分别与左、右括号相对应, 则长度为 n 的栈混洗, 必然与由 n 对括号组成的合法表达式彼此对应 (习题 [4-4])。比如, 栈混洗 { 3, 2, 4, 1 } 对应于表达式 "((()) ())"。按照这一理解, 借助栈结构, 只需扫描一趟表达式, 即可在线性时间内, 判定其中的括号是否匹配。这一新的算法, 可简明地实现如代码 4.5 所示。

```
1 bool paren(const char exp[], int lo, int hi) { //表达式括号匹配检查, 可兼顾三种括号
2     Stack<char> S; //使用栈记录已发现但尚未匹配的左括号
3     for (int i = 0; exp[i]; i++) /* 逐一检查当前字符 */
4         switch (exp[i]) { //左括号直接进栈; 右括号若与栈顶失配, 则表达式必不匹配
5             case '(': case '[': case '{': S.push(exp[i]); break;
6             case ')': if ((S.empty() || '(' != S.pop())) return false; break;
7             case ']': if ((S.empty() || '[' != S.pop())) return false; break;
8             case '}': if ((S.empty() || '{' != S.pop())) return false; break;
9             default: break; //非括号字符一律忽略
10        }
11    return S.empty(); //整个表达式扫描过后, 栈中若仍残留 (左) 括号, 则不匹配; 否则 (栈空) 匹配
12 }
```

代码4.5 括号匹配算法 (迭代版)

新算法的流程控制简单, 而且便于推广至多类括号并存的情况。它自左向右逐个考查各字符, 忽略所有非括号字符。凡遇到左括号, 无论属于哪类均统一压入栈 S 中。若遇右括号, 则弹出栈顶的左括号并与之比对。若二者属于同类, 则继续检查下一字符; 否则, 即可断定表达式不匹配。当然, 栈 S 提前变空或者表达式扫描过后栈 S 非空, 也意味着不匹配。

图 4.6 给出了一次完整的计算过程。表达式扫描完毕时, 栈 S 恰好为空, 故知表达式匹配。

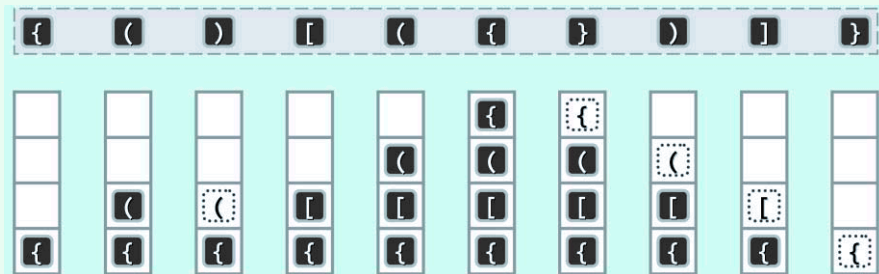


图4.6 迭代式括号匹配算法实例

(上方为输入表达式; 下方为辅助栈的演变过程; 虚框表示在 (右) 括号与栈顶 (左) 括号匹配时对应的出栈操作)

4.3.3 延迟缓冲

在一些应用问题中，输入可分解为多个单元并通过迭代依次扫描处理，但过程中的各步计算往往滞后于扫描的进度，需要待到必要的信息已完整到一定程度之后，才能作出判断并实施计算。在这类场合，栈结构则可以扮演数据缓冲区的角色。

■ 表达式求值

在编译C++程序的预处理阶段，源程序中的所有常量表达式都需首先计算并替换为对应的具体数值。而在解释型语言中，算术表达式的求值也需随着脚本执行过程中反复进行。

比如，在UNIX Shell、DOS Shell和PostScript交互窗口中分别输入：

```
$ echo $(( 0 + ( 1 + 23 ) / 4 * 5 * 67 - 8 + 9 ))
\> set /a (( 0 + ( 1 + 23 ) / 4 * 5 * 67 - 8 + 9 ))
GS> 0 1 23 add 4 div 5 mul 67 mul add 8 sub 9 add =
```

都将返回“2011”。

可见，不能简单地按照“先左后右”的次序执行表达式中的运算符。关于运算符执行次序的规则（即运算优先级），一部分决定于事先约定的惯例（比如乘除优先于加减），另一部分则决定于括号。也就是说，仅根据表达式的某一前缀，并不能完全确定其中各运算符可否执行以及执行的次序；只有在已获得足够多后续信息之后，才能确定其中哪些运算符可以执行。

■ 优先级表

我们首先如代码4.6所示，将不同运算符之间的运算优先级关系，描述为一张二维表格。

```
1 #define N_OPTR 9 //运算符总数
2 typedef enum {ADD, SUB, MUL, DIV, POW, FAC, L_P, R_P, EOE} Operator; //运算符集合
3 //加、减、乘、除、乘方、阶乘、左括号、右括号、起始符与终止符
4
5 const char pri[N_OPTR][N_OPTR] = { //运算符优先等级 [栈顶] [当前]
6     /*          |----- 当前运算符 -----| */
7     /*          +   -   *   /   ^   !   (   )   \0 */
8     /* -- + */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
9     /* | - */ '>', '>', '<', '<', '<', '<', '<', '>', '>',
10    /* 栈 * */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
11    /* 顶 / */ '>', '>', '>', '>', '<', '<', '<', '>', '>',
12    /* 运 ^ */ '>', '>', '>', '>', '>', '<', '<', '>', '>',
13    /* 算 ! */ '>', '>', '>', '>', '>', '>', ' ', '>', '>',
14    /* 符 ( */ '<', '<', '<', '<', '<', '<', '<', '=', ' ',
15    /* | ) */ ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
16    /* -- \0 */ '<', '<', '<', '<', '<', '<', '<', ' ', '='
17 };
```

代码4.6 运算符优先级关系的定义

在常规的四则运算之外，这里还引入了乘方和阶乘运算。其中阶乘属于一元运算，且优先级最高。为统一算法的处理流程，将左、右括号以及标识表达式尾部的字符'\0'，也视作运算符。

■ 求值算法

基于运算符优先级如上的定义和判定规则，可实现表达式求值算法如代码4.7所示。

```
1 float evaluate(char* S, char*& RPN) { //对 (已剔除空白格的) 表达式S求值, 并转换为逆波兰式RPN
2     Stack<float> opnd; Stack<char> optr; //运算数栈、运算符栈
3     optr.push('\0'); //尾哨兵'\0'也作为头哨兵首先入栈
4     while (!optr.empty()) { //在运算符栈非空之前, 逐个处理表达式中各字符
5         if (isdigit(*S)) { //若当前字符为操作数, 则
6             readNumber(S, opnd); append(RPN, opnd.top()); //读入操作数, 并将其接至RPN末尾
7         } else //若当前字符为运算符, 则
8             switch(orderBetween(optr.top(), *S)) { //视其与栈顶运算符之间优先级高低分别处理
9                 case '<': //栈顶运算符优先级更低时
10                     optr.push(*S); S++; //计算推迟, 当前运算符进栈
11                     break;
12                 case '=': //优先级相等 (当前运算符为右括号或者尾部哨兵'\0') 时
13                     optr.pop(); S++; //脱括号并接收下一个字符
14                     break;
15                 case '>': { //栈顶运算符优先级更高时, 可实施相应的计算, 并将结果重新入栈
16                     char op = optr.pop(); append(RPN, op); //栈顶运算符出栈并续接至RPN末尾
17                     if ('!' == op) { //若属于一元运算符
18                         float pOpnd = opnd.pop(); //只需取出一个操作数, 并
19                         opnd.push(calcu(op, pOpnd)); //实施一元计算, 结果入栈
20                     } else { //对于其它 (二元) 运算符
21                         float pOpnd2 = opnd.pop(), pOpnd1 = opnd.pop(); //取出后、前操作数
22                         opnd.push(calcu(pOpnd1, op, pOpnd2)); //实施二元计算, 结果入栈
23                     }
24                     break;
25                 }
26                 default : exit(-1); //逢语法错误, 不做处理直接退出
27             } //switch
28     } //while
29     return opnd.pop(); //弹出并返回最后的计算结果
30 }
```

代码4.7 表达式的求值及RPN转换

该算法自左向右扫描表达式, 并对其中字符逐一做相应的处理。那些已经扫描过但 (因信息不足) 尚不能处理的操作数与运算符, 将分别缓冲至栈opnd和栈optr。一旦判定已缓存的子表达式优先级足够高, 便弹出相关的操作数和运算符, 随即执行运算, 并将结果压入栈opnd。

请注意这里区分操作数和运算符的技巧。一旦当前字符由非数字转为数字, 则意味着开始进入一个对应于操作数的子串范围。由于这里允许操作数含有多个数位, 甚至可能是小数, 故可用readNumber()函数 (习题[4-6]), 根据当前字符及其后续的若干字符, 利用另一个栈解析出当前的操作数。解析完毕, 当前字符将再次聚焦于一个非数字字符。

■ 不同优先级的处置

按照代码4.7，若当前字符为运算符，则在调用`orderBetween()`函数（习题[4-7]），将其与栈`optr`的栈顶操作符做一比较之后，即可视二者的优先级高低，分三种情况相应地处置。

1) 若当前运算符的优先级更高，则`optr`中的栈顶运算符尚不能执行

以表达式“`1 + 2 * 3 ...`”为例，在扫描到运算符`*`时，`optr`栈顶运算符为此前的`+`，由于`pri['+']['*'] = '<'`，当前运算符`*`优先级更高，故栈顶运算符`+`的执行必须推迟。

请注意，由代码4.6定义的优先级表，无论栈顶元素如何，当前操作符为`(`的所有情况均统一归入这一处理方式；另外，无论当前操作符如何，栈顶操作符为`(`的所有情况也统一按此处理。也就是说，所有左括号及其后紧随的一个操作符都会相继地被直接压入`optr`栈中，而此前的运算符则一律押后执行——这与左括号应有的功能完全吻合。

2) 反之，一旦栈顶运算符的优先级更高，则可以立即弹出并执行对应的运算

以表达式“`1 + 2 * 3 - 4 ...`”为例，在扫描到运算符`-`时，`optr`栈顶运算符为`*`，由于`pri['*']['-'] = '>'`，意味着当前运算符的优先级更低，故栈顶运算符`*`可立即执行。

类似地，根据代码4.6定义的优先级表，无论栈顶元素如何，当前操作符为`)`的情况也几乎全部归入这一处理方式。也就是说，一旦抵达右括号，此前在`optr`栈缓冲的运算符大都可以逐一弹出并执行——这与右括号应有的功能也完全吻合。

3) 当前运算符与栈顶运算符的优先级“相等”

对右括号的上述处理方式，将在`optr`栈顶出现操作符`(`时终止——由代码4.6可知，`pri['('][')'] = '='`。此时，将弹出栈顶的`(`，然后继续处理`)`之后的字符。不难看出，这对左、右括号在表达式中必然相互匹配，其作用在于约束介乎二者之间的那段子表达式的优先级关系，故在其“历史使命”完成之后，算法做如上处置理所应当。

除左、右括号外，还有一种优先级相等的合法情况，即`pri['\0']['\0'] = '='`。由于在算法启动之初已经首先将字符`\0`压入`optr`栈，故在整个表达式已被正确解析并抵达表达式结束标识符`\0`时，即出现这一情况。对于合法的表达式，这种情况只在算法终止前出现一次。既然同是需要弹出栈顶，算法不妨将这种情况按照优先级相等的方式处置。

■ 语法检查及鲁棒性

为简洁起见，以上算法假设输入表达式的语法完全正确；否则，有可能会導致荒诞的结果。读者可在此基础上，尝试扩充语法检查以及对各种非法情况的处理功能（习题[4-12]）。

4.3.4 逆波兰表达式

■ RPN

逆波兰表达式（reverse Polish notation, RPN）是数学表达式的一种，其语法规则可概括为：操作符紧邻于对应的（最后一个）操作数之后。比如“`1 2 +`”即通常习惯的“`1 + 2`”。

按此规则，可递归地得到更复杂的表达式，比如RPN表达式

`1 2 + 3 4 ^ *`

即对应于常规的表达式

`( 1 + 2 ) * 3 ^ 4`